

LoSemB: Logic-Guided Semantic Bridging for Inductive Tool Retrieval

Luyao Zhuang
luyao.zhuang@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Qinggang Zhang*
qinggangg.zhang@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Huachi Zhou
huachi.zhou@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Yujing Zhang
yu-jing.zhang@connect.polyu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Xiao Huang
xiao.huang@polyu.edu.hk
The Hong Kong Polytechnic
University
Hong Kong SAR, China

Abstract

Equipping large language models (LLMs) with external tools has emerged as a promising paradigm for addressing real-world tasks. Nonetheless, with the web-based tool ecosystems rapidly expanding, it is impractical to include all tools within the limited input length of LLMs. To alleviate these issues, researchers have explored incorporating a tool retrieval module to select the most relevant tools or represent tools as unique tokens within LLM parameters. However, most state-of-the-art methods are under transductive settings, assuming all tools have been observed during training. Such a setting deviates from reality as tools on the web are constantly updated and new tools are frequently added to the online ecosystem. When dealing with these unseen tools, which refer to tools not encountered during the training phase, these methods are limited by two key issues, including the large distribution shift and the sensitivity of semantic-only retrieval. To this end, inspired by human cognitive processes of mastering unseen tools through discovering and applying the logical information from prior experience, we introduce a novel **Logic-Guided Semantic Bridging** framework for inductive tool retrieval, namely, **LoSemB**, which aims to mine and transfer latent logical information for inductive tool retrieval without costly retraining. Specifically, LoSemB contains a logic-based embedding alignment module to mitigate distribution shifts and a relational augmented retrieval mechanism to overcome the limitations of semantic-only similarity methods. Extensive experiments demonstrate that LoSemB achieves advanced performance in both the inductive and transductive settings.

CCS Concepts

• Information systems → Information retrieval.

*Corresponding author



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792336>

Keywords

Web API, Tool Retrieval, Semantics Mining, Language Models

ACM Reference Format:

Luyao Zhuang, Qinggang Zhang, Huachi Zhou, Yujing Zhang, and Xiao Huang. 2026. LoSemB: Logic-Guided Semantic Bridging for Inductive Tool Retrieval. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792336>

1 Introduction

While large language models (LLMs) [1, 2, 25, 45] have demonstrated remarkable capabilities across diverse tasks [22, 56], they still fall short in some problems, *e.g.* complex computations and providing real-time information [44], due to their reliance on fixed and parametric knowledge [38]. Recently, to extend the abilities of LLMs, tool learning [4, 5, 28, 29, 37], which augments LLMs with external tools, has attracted enormous attention. For instance, web search engines enable LLMs to access real-time information, overcoming temporal limitations and providing informative responses.

However, as the number of tools equipped with LLMs increases to the tens of thousands, it has become challenging to contain all the tools and their descriptions [20], hindered by the limited context length of LLMs [3, 26]. To tackle this issue, two primary research directions have emerged, as depicted in Figure 1: 1) Token-based methods [16, 41, 47] represent each tool as a specific token and integrate tool knowledge directly into the LLM's parameters. Through fine-tuning processes that memorize each tool document and map it to its corresponding token, these models develop the capability to generate appropriate tool calls autonomously. 2) Retrieval-based methods [37, 55, 60] retrieve relevant tools from the large tool repository by calculating the similarity between the instruction and the tools. These methods employ either sparse lexical similarity retrievers like BM25 [40], which require no training, or dense embedding retrievers that utilize pre-trained language models (PLMs). Dense retrievers [61] can be further categorized into training-free approaches, such as Re-Invoke [9], which rewrites instructions and extracts their intents to enhance tool documentation, and training-based approaches like ToolRetriever [36], which fine-tunes on instruction-tool pairs to optimize retrieval performance.

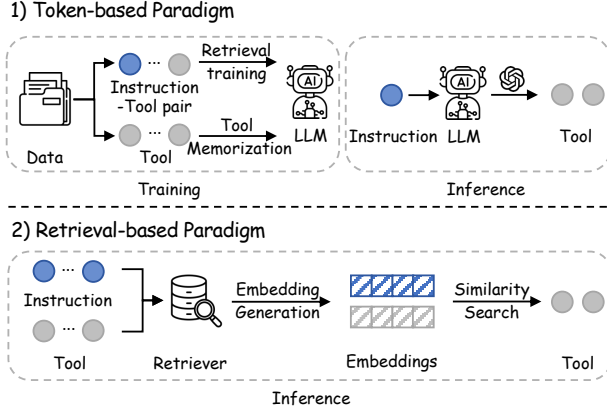


Figure 1: Token-based vs. Retrieval-based paradigm.

In this regard, most state-of-the-art methods require domain-specific training and operate under transductive settings, where all tools are available during the training phase. However, real-world scenarios frequently involve new tools or the addition of new functionalities to existing tools, posing challenges for both paradigms. For token-based methods, the core mechanism relies on learning compressed token representations that associate each tool with its usage contexts through training. When encountering unseen tools, these methods become completely ineffective unless the entire model is retrained to learn new token associations. On the other hand, retrieval-based methods struggle to generate accurate representations for unseen tools, as evidenced by our preliminary experiments showing large performance degradation for fine-tuned retrievers in the inductive setting.

Based on the experimental results, we identified two challenges in retrieval-based methods when handling unseen tools. **❶ Large Distribution Shift.** Our analysis reveals that unseen tools show large distribution shifts due to functional diversity across different tools. This causes retrievers trained on seen tools to learn biased representations that fail to capture the true functionality of unseen tools, ultimately resulting in retrieval failures. **❷ Sensitivity of Semantic-only Retrieval.** Traditional retrieval-based methods primarily rely on computing semantic similarity between instructions and tool descriptions. However, this approach is highly sensitive to the quality of learned representations. When dealing with unseen tools, their representations suffer from distribution shifts, leading to worse retrieval performance. Drawing from practical experience, we propose and validate that tools typically exhibit sparse co-occurrence relationships, and semantically similar instructions often correspond to overlapping tool sets. This logical information provides opportunities to mitigate distribution shifts and achieve more robust retrieval.

To this end, we draw inspiration from how humans adapt to unseen tools. When encountering unseen tools, humans first systematically organize their existing knowledge, identifying the relationships between tools and their usage scenarios, as well as the functional connections among tools. Through this structured analysis, humans extract deeper logical information and then transfer it to guide their understanding and use of the unseen tool [13, 50]. This cognitive process reveals that logical information plays a crucial role in adapting to unseen tools. Thus, our framework is guided

by two critical questions: **❶** How can we efficiently extract hidden logical information and transfer it to learn better representations for unseen tools? **❷** How can we effectively integrate logical information into the retrieval paradigm, rather than relying only on semantic similarity, to guide an accurate tool retrieval process? Our main contributions are summarized as follows:

- We identify the key challenges of existing tool retrieval methods in the inductive setting and propose LoSemB to improve the accuracy of retrieving unseen tools without costly retraining.
- LoSemB introduces a novel logic-based embedding alignment module that addresses the distribution shift by integrating logical features into the representations of unseen tools.
- Building upon logically enhanced embeddings, LoSemB further adopts a relational augmented retrieval mechanism that leverages both logical pruning and similarity of the enhanced-embeddings to overcome the sensitivity of semantic-only retrieval.
- Experiments show that LoSemB achieves advanced performance in inductive settings while maintaining desirable effectiveness in the transductive setting.

2 Problem Statement

Given a test instruction q_{test} , the tool retrieval task requires identifying a tool set $\mathcal{T}_{ret} \subseteq \mathcal{T} = \{t_j\}_{j=1}^m$ with the highest relevance scores calculated by a retrieval function $f(\cdot)$. In this paper, we delve into the impact of both transductive and inductive settings on tool retrieval. The transductive setting assumes all tools in the repository are seen during training, where we use instruction-tool pairs to finetune the retriever. However, in practice, tool repositories are frequently updated with new tools that were unavailable during training. We define this scenario as the inductive tool retrieval setting. Therefore, when training the retriever for an inductive setting, we must filter the training dataset to exclude the unseen tools and their corresponding unseen instructions to properly evaluate tool retrieval performance in the inductive setting.

3 Preliminary Study

Before going into the technical details of LoSemB, we first conduct a preliminary study to identify the primary challenges of the retrieval-based method in the inductive setting.

3.1 Performance Degradation

While retrieval-based methods show promise for tool learning, their effectiveness in handling unseen tools remains unclear. To investigate this, we evaluate fine-tuned retrievers in the inductive setting. As shown in Figure 2 (a), we trained a BERT-base [10] retriever on the I2 and I3 training sets of ToolBench and tested it with varying proportions of unseen tools (10%, 20%, and 30%). Our experiments reveal that retrieval performance declines as the proportion of unseen tools increases. Notably, the I3 dataset showed significant degradation, with relative accuracy drops of 4.63%, 13.77%, and 21.70% for the three unseen tool ratios, respectively.

3.2 Cause Analysis

To understand what underlying challenges lead to this performance degradation, we conduct analyses about the representations of

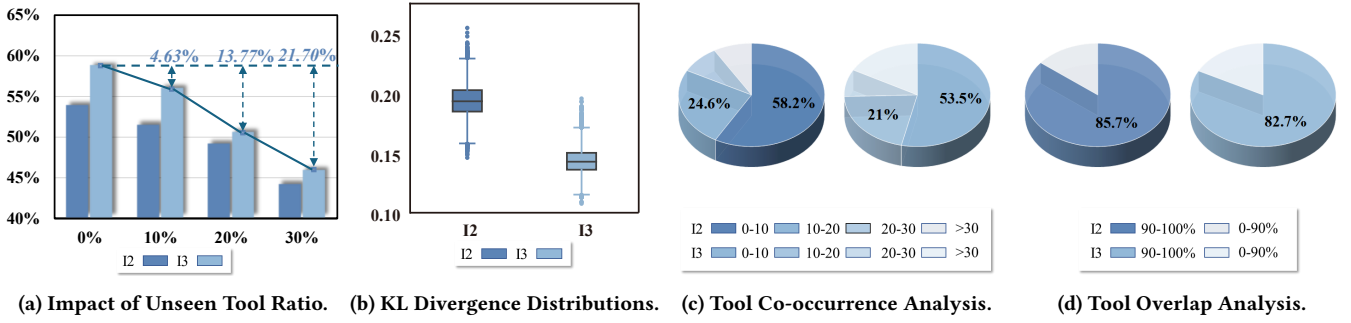


Figure 2: (a) Impact of unseen tool ratio on retrieval performance, (b) Comparison of KL divergence distributions between seen and unseen tools, (c) Distribution of tool co-occurrence counts across datasets, (d) Tool overlap percentages between each instruction’s tool set and its Top-5 semantically similar instructions. All results are based on ToolBench (I2) and (I3).

Table 1: Retrieval error case illustrating limitations of the semantic-only method on unseen tools.

Instruction: I’m planning a family gathering and I want to make some non-alcoholic cocktails for the kids. Can you suggest some fun and colorful mocktail recipes? Also, provide me with the contact details of a brewery that offers non-alcoholic beer options.
Semantic-only retrieval: [Food, Tasty, recipes/detail]
Label: [Food, The Cocktail DB, Detailed Cocktail Recipe by ID]

unseen tools and the impact of the existing retrieval architecture, as these two critical factors directly determine retrieval performance.

Challenge ①: Large Distribution Shift. As shown in Figure 2 (b), we measured the KL divergence between the representation distributions of unseen tools v.s. seen tools (tools encountered during training). We observe that unseen tools exhibit large distributional divergence from seen tools. To understand the impact of this distribution shift on retrieval performance, we analyze it through the lens of domain adaptation theory [39]. Let $\mathcal{P}_{\text{seen}}$ denote the distribution of seen tools, and $\mathcal{P}_{\text{unseen}}$ denote the distribution of unseen tools. For any retrieval function $g(\cdot)$, the retrieval error on unseen tools is bounded by:

$$\epsilon_{\text{unseen}}(g(\cdot)) \leq \epsilon_{\text{seen}}(g(\cdot)) + \text{KL}(\mathcal{P}_{\text{unseen}} \parallel \mathcal{P}_{\text{seen}}), \quad (1)$$

where $\epsilon_{\text{unseen}}(g(\cdot))$ represents the expected retrieval error on unseen tools; $\epsilon_{\text{seen}}(g(\cdot))$ represents the retrieval error on seen tools; and $\text{KL}(\mathcal{P}_{\text{unseen}} \parallel \mathcal{P}_{\text{seen}})$ quantifies the KL divergence between the distributions of unseen and seen tools, measuring the degree of distribution shift. This reveals that directly encoding unseen tool representations using models trained on seen tools enlarges the error bound. Consequently, this results in biased representations fail to capture the true functionality of unseen tools, leading to retrieval failures in the inductive setting.

Challenge ②: Sensitivity of Semantic-only Retrieval. Traditional retrieval-based methods primarily rely on computing semantic similarity between instructions and tool descriptions. However, this approach is highly sensitive to the quality of learned representations. When dealing with unseen tools, their representations often suffer from distribution shifts, causing semantic similarity calculations to produce severely misleading results.

From the example in Table 1, consider the unseen tool “The Cocktail DB” in ToolBench. Given an instruction about “mocktail recipes” for a family gathering, this tool should ideally be retrieved, as it specializes in cocktail recipe services. However, because “The Cocktail DB” is an unseen tool not present in the training data, its representation suffers from distribution shift, causing it to be incorrectly positioned in the embedding space. As a result, semantic similarity calculations on this biased representation retrieve the “recipes/detail” tool, which matches the “recipes” keyword but lacks cocktail-specific functionality.

3.3 Discussion and Insights

As analyzed above, traditional methods rely only on semantic information, making them sensitive when handling unseen tools. These insights inform our revisit of the retrieval paradigm. Beyond semantic information, we need to identify other potential information.

Drawing from practical experience, we notice two intuitive patterns in tool usage: (i) Each tool tends to work with a relatively fixed and small set of companion tools. For example, when users book a flight, they typically need flight search, price comparison, booking, and payment tools working together. A payment tool consistently appears alongside booking tools, but rarely with unrelated tools like weather forecasting. (ii) When users express similar needs using different phrasings, such as “check my account balance” versus “view my current funds,” they essentially require the same set of tools despite the semantic variations.

To validate these intuitions, we conduct an empirical analysis on the I2 and I3 subsets of the ToolBench dataset. As shown in Figure 2 (c), each tool maintains a limited set of co-occurrence partners. Specifically, in the ToolBench (I2) dataset, 58.2% of tools co-occur with fewer than 10 other tools, while an additional 24.6% co-occur with only 10-20 tools. And, refer to Figure 2 (d), for each instruction, we identify the Top-5 similar instructions and find that, on average, 85.7% of tools corresponding to each instruction overlap with the tool set associated with its similar instructions. In conclusion, we can leverage logical information from tool relationships and usage patterns to learn more accurate representations for unseen tools and overcome the sensitivity of semantic-only retrieval.

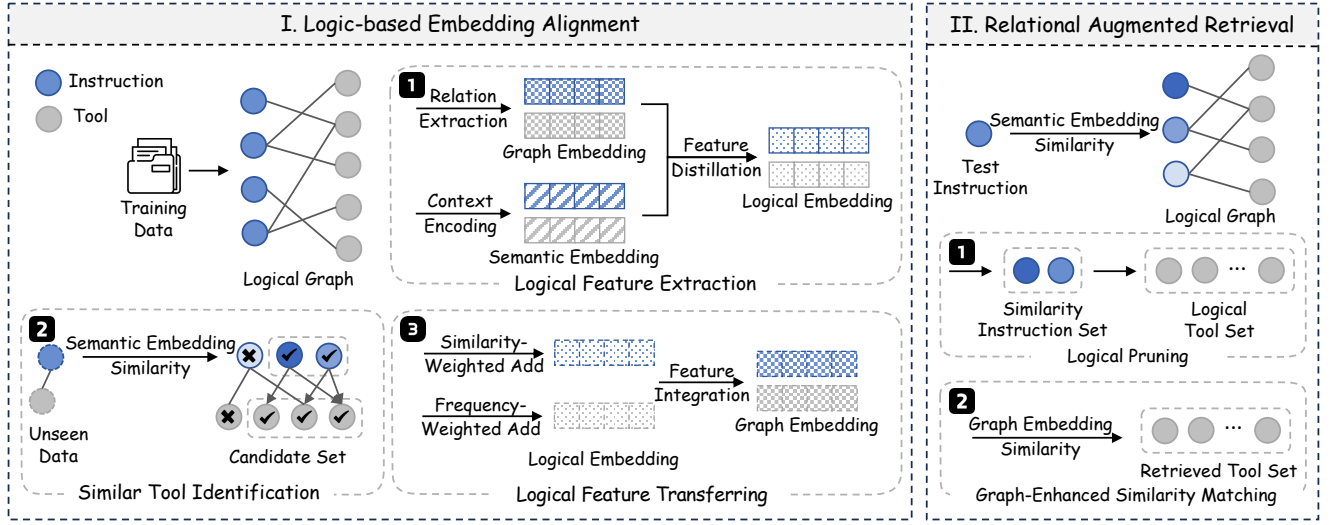


Figure 3: The overall framework of LoSemB.

4 LoSemB Framework

4.1 Overview

Based on our preliminary findings, logical information serves as a crucial component for addressing the challenges in inductive tool retrieval. Accordingly, we design a two-stage framework comprising offline indexing and online retrieval, as illustrated in Figure 3. In the offline indexing phase, we first construct a graph containing tool nodes and instruction nodes. Building upon this graph, we introduce a logic-based embedding alignment module that extracts latent logical features and transfers them to unseen tools. In the online retrieval phase, we implement a relational augmented retrieval module. This module leverages the encoded logical relationships from the graph to guide the retrieval process by combining logical pruning with enhanced similarity matching, ultimately identifying the most relevant tools.

4.2 Logical Graph Construction

Given a corpus with a set of n instructions $Q = \{q_1, \dots, q_n\}$ and a tool repository $\mathcal{T} = \{t_1, \dots, t_m\}$ containing m tools, all of which are available during retriever training. Then, we construct a bipartite graph, called the logical graph, denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{D})$, where $\mathcal{V} = Q \cup \mathcal{T}$ and $\mathcal{D}$ contains the textual descriptions associated with the nodes in $\mathcal{V}$.

The edges $\mathcal{E}$ are constructed according to the following rule: if an instruction q_i requires a tool t_j to satisfy its request, we add an edge (v_{q_i}, v_{t_j}) . These relations correspond to an incidence matrix, called the *invoke matrix* $\mathbf{R}$, between instructions and tools.

Formally, the invoke matrix $\mathbf{R}$ is defined as an $n \times m$ matrix:

$$\mathbf{R} = [r_{ij}]_{n \times m}; r_{ij} = \mathbb{1}(q_i \leftrightarrow t_j). \quad (2)$$

Here, r is the entry of matrix $\mathbf{R}$; $\leftrightarrow$ represents a bidirectional invocation relation from instruction to tool; $\mathbb{1}$ denotes the indicator function, where $r_{ij} = 1$ if instruction q_i invokes tool t_j ($q_i \leftrightarrow t_j$), and $r_{ij} = 0$ if there is not invocation relation.

We encode the textual content of instructions and the descriptions of tools to generate initial embeddings using a fine-tuned pre-trained language model (PLM). Specifically, we obtain the instruction embeddings $\mathbf{E}_q^{(0)} \in \mathbb{R}^{n \times d}$ and the tool embeddings $\mathbf{E}_t^{(0)} \in \mathbb{R}^{m \times d}$. We refer to these initial embeddings as semantic embeddings. Then we concatenate them as $\mathbf{E}^{(0)} = \mathbf{E}_q^{(0)} \oplus \mathbf{E}_t^{(0)} \in \mathbb{R}^{(n+m) \times d}$, where d denotes the embedding dimension.

4.3 Logic-based Embedding Alignment

Our analysis in Section 3.2 reveals that unseen tools suffer from the large distribution shift, causing degraded retrieval performance. Our key insight is that when tools have similar functions, they will demonstrate similar instruction-tool invoke patterns and tool co-occurrence relationships in the graph. Therefore, based on this logical information, we can extract logical features from functionally similar seen tools and transfer them to unseen tools.

However, it raises several critical questions, *i.e.*, how to extract latent logical features, how to select nodes from which to transfer features, and how to transfer them to unseen nodes. We address these through three stages: logical feature extraction, similar tool identification, and logical feature transfer.

Logical Feature Extraction. We first need to extract logical features from existing nodes. Our approach extracts logical features by capturing how graph convolution transforms the original semantic embeddings by incorporating the logical information of the instruction-tool invoke pattern and tool co-occurrence relationships. As a result, during graph convolution, nodes change from their semantic embeddings, and then we can capture the logical features by finding the difference between the graph embeddings and the original embeddings.

Specifically, we employ a multi-layer graph convolutional mechanism, following LightGCN [17],

$$\mathbf{E}^{(l+1)} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{E}^{(l)}, \quad (3)$$

where $\mathbf{A} = \begin{bmatrix} 0_{n \times n} & \mathbf{R} \\ \mathbf{R}^T & 0_{m \times m} \end{bmatrix}$ is the adjacency matrix constructed from the invoke matrix $\mathbf{R}$, and $\mathbf{D}$ is the diagonal degree matrix with $d_{ii} = \sum_j a_{ij}$, where a_{ij} is the entry of $\mathbf{A}$.

After L layers of propagation, we merge embeddings from all layers to form the final graph embeddings:

$$\mathbf{E} = \frac{1}{L+1} \sum_{l=0}^L \mathbf{E}^{(l)} \quad (4)$$

We then perform feature distillation through a comparative transformation that maps graph embeddings against original semantic embeddings to obtain logical embeddings:

$$\Delta \mathbf{E} = \mathbf{E} - \mathbf{E}^{(0)}. \quad (5)$$

Here, Δ represents the logical feature matrix, where $\Delta \mathbf{E}_{t_j}$ and $\Delta \mathbf{E}_{q_i}$ correspond to the logical features of tool and instruction nodes. Thus, this approach provides three complementary representations for each node: semantic embeddings contain content information, graph embeddings combine logical and semantic content, and logical features capture the node's functional role within the graph.

Similar Tool Identification via Structured Propagation. After extracting logical features, we need to identify which existing nodes contain the most similar logical features for unseen tools. While intuitively we might search for textually similar tools as sources for knowledge transfer, this approach performs poorly when the retriever struggles to generate accurate representations for unseen tools. Moreover, textually similar tools do not necessarily share functional similarities. Our key insight is that tools that process similar instructions are more likely to share similar functions and logical features. Therefore, we use the unseen tool's associated instruction as a bridge to locate functionally similar tools through a structured propagation mechanism.

Formally, given a set of unseen tools $\mathcal{T}^u = \{t_1^u, \dots, t_h^u\}$, for each unseen tool t^u with its associated instruction q^u , we first encode them using the same PLM to obtain their initial semantic embeddings $\mathbf{e}_{t^u}^{(0)}$ and $\mathbf{e}_{q^u}^{(0)}$. We then compute the contextual association between the unseen instruction q^u and all training instructions $\mathcal{Q}$ based on their semantic embeddings, representing these similarities as a vector:

$$\mathbf{s} = [s_i]_{n \times 1}; s_i = \cos(\mathbf{e}_{q_i}^{(0)}, \mathbf{e}_{q^u}^{(0)}). \quad (6)$$

This vector $\mathbf{s}$ captures how similar each training instruction $q \in \mathcal{Q}$ is to the unseen instruction q^u , providing a comprehensive similarity distribution across the entire training instruction space. To identify the most relevant instruction candidates while filtering out noise, we apply a top- J selection mechanism that constructs a sparse activation vector $\mathbf{c} = [c_i]_{n \times 1}$, where $c_i = \mathbb{1}(i \in \text{Top-}J(\mathbf{s})) \cdot s_i$. Here $\mathbb{1}$ is the indicator function and $\text{Top-}X(\cdot)$ operator returns the indices of the top X elements of the input set. We then propagate this sparse instruction activation through the invoke matrix $\mathbf{R}$ to aggregate activation scores for candidate tools via matrix multiplication:

$$\mathbf{b} = \mathbf{R}^T \mathbf{c} \in \mathbb{R}^{m \times 1}, \quad (7)$$

where $\mathbf{b} = [b_j]_{m \times 1}$ represents the aggregated activation scores for all training tools $\mathcal{T}$. Each element b_j quantifies the functional similarity between unseen tool t^u and training tool t_j , computed by aggregating the activation weights from all similar instructions that invoke t_j . Intuitively, if training tool t_j is invoked by some

instructions similar to q^u , it receives a higher activation score, suggesting that t_j serves similar functional roles as the unseen tool t^u . We identify tools with positive activation scores as functionally similar candidates: $\mathcal{T}_{\text{cand}} = \{t_j | b_j > 0, t_j \in \mathcal{T}\}$.

Logical Feature Transfer. After identifying candidate nodes, we design tailored weighting mechanisms to prioritize the most valuable logical features from candidate nodes. For unseen tools, we employ a frequency-based weighting strategy motivated by the insight that tools frequently invoked by similar instructions are likely to share similar logical patterns. Specifically, the weight for each candidate tool $t_j \in \mathcal{T}_{\text{cand}}$ is computed by normalizing its invocation frequency among the top- J similar instructions:

$$w_j = \frac{\sum_{i=1}^n r_{ij} \cdot \mathbb{1}_{\{c_i > 0\}}}{\sum_{t_p \in \mathcal{T}_{\text{cand}}} \sum_{i=1}^n r_{ip} \cdot \mathbb{1}_{\{c_i > 0\}}}, \quad (8)$$

where r_{ij} is the entry of invoke matrix, and $\mathbb{1}_{\{c_i > 0\}}$ indicates whether instruction q_i is among the top- J similar instructions. Using these weights, we apply feature integration to generate the graph embedding for the unseen tool t^u by combining its semantic embedding with the weighted logical features:

$$\mathbf{e}_{t^u} = \mathbf{e}_{t^u}^{(0)} + \sum_{t_j \in \mathcal{T}_{\text{cand}}} w_j \cdot \Delta \mathbf{e}_{t_j}. \quad (9)$$

For unseen instructions, we adopt a similarity-based softmax normalization strategy to ensure more semantically similar instructions contribute more significantly to the feature transfer. The weight for each selected training instruction is calculated as:

$$w_i = \frac{\exp(s_i)}{\sum_{q_i: c_i > 0} \exp(s_i)}, \quad (10)$$

where s_i is the cosine similarity and the denominator only includes instructions with non-zero activation. We then generate the graph embedding for the unseen instruction q^u with the weighted logical features:

$$\mathbf{e}_{q^u} = \mathbf{e}_{q^u}^{(0)} + \sum_{q_i: c_i > 0} w_i \cdot \Delta \mathbf{e}_{q_i}. \quad (11)$$

Finally, by aligning unseen nodes with existing nodes through weighted logical feature transfer, we obtain better representations of unseen tools without requiring costly retraining, and then seamlessly integrate them into the logical graph $\mathcal{G}$.

4.4 Relational Augmented Retrieval

Our key insight is if certain tools are frequently invoked by similar training instructions, they are likely relevant to semantically related test instructions. Building on this observation, we leverage the logical graph to exploit such relational patterns. Since our approach relies on observed instruction-tool relationships rather than solely on semantic representations of tools, it is less sensitive to representation quality. Furthermore, we refine similarity matching using graph-enhanced embeddings that incorporate logical information.

Logical Pruning. Given a test instruction q^{test} , we first compute its similarity distribution with all training instructions:

$$\mathbf{s}^{\text{test}} = [s_i^{\text{test}}]_{n \times 1}; s_i^{\text{test}} = \cos(\mathbf{e}_{q_i}^{(0)}, \mathbf{e}_{q^{\text{test}}}^{(0)}), \quad (12)$$

where $q_i \in \mathcal{Q}$ is the training instruction set. We then identify the top- I most similar instructions and collect all tools invoked by them

Table 2: Results (%) of baselines and LoSemB in transductive and inductive settings across five datasets. We report the average performance across R@3, R@7, P@3, and P@7. The best result for each dataset is highlighted in bold, and the second-best is indicated with an underline.

Method	ToolBench (I1)		ToolBench (I2)		ToolBench (I3)		UtralTool		APIGen	
	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive
<i>Direct Zero-shot LLM Inference</i>										
GPT-3.5-turbo	1.31	1.31	1.91	1.91	10.79	10.79	6.51	6.51	3.95	3.95
GPT-4o-mini	10.45	10.45	8.67	8.67	48.26	48.26	29.55	29.55	18.50	18.50
<i>Training-free Tool Retrieval Method</i>										
BM25	17.37	17.37	13.29	13.29	15.77	15.77	9.22	9.22	34.60	34.60
Ada Embedding	45.90	45.90	26.34	26.34	34.75	34.75	26.56	26.56	39.00	39.00
all-MiniLM-L6-v2	32.75	32.75	17.76	17.76	23.90	23.90	19.45	19.45	34.40	34.40
Re-invoke	38.06	38.06	30.69	30.69	37.25	37.25	14.17	14.17	32.00	32.00
<i>Training-based Tool Retrieval Methods</i>										
EmbSim	64.56	58.75	53.99	44.35	58.86	46.09	35.53	26.87	45.65	43.60
InstRet	<u>66.35</u>	<u>61.93</u>	<u>56.39</u>	44.70	60.68	48.62	36.11	29.54	46.00	44.00
RerankRet	65.78	60.74	53.77	<u>50.15</u>	<u>68.03</u>	<u>54.35</u>	<u>61.74</u>	<u>48.83</u>	<u>48.85</u>	<u>46.20</u>
LoSemB (Ours)	70.46	68.44	66.37	64.41	71.47	68.48	67.55	65.35	60.20	58.45

as the pruned candidate set:

$$\mathcal{T}_{\text{logic}} = \{t_j | (q_i, t_j) \in \mathcal{E}, i \in \text{Top-}I(s^{\text{test}})\}. \quad (13)$$

This logic-based pruning effectively reduces the search space from m tools to $|\mathcal{T}_{\text{logic}}| \ll m$ by focusing on tools that have demonstrated usefulness for similar instructions, leveraging the structural patterns captured in the logical graph $\mathcal{G}$.

Graph-Enhanced Similarity Matching. Within the pruned candidate set $\mathcal{T}_{\text{logic}}$, we perform graph-enhanced similarity matching that incorporates both semantic and logical information to achieve more accurate similarity matching. We first generate the graph embedding for the test instruction $\mathbf{e}_{q^{\text{test}}}$ following the same feature transfer process described earlier. (Note that during this process, we fuse the logical features from the top- $I(s)$ most similar instructions in the training set.). Then, we retrieve the top- K most relevant tools by ranking the cosine similarity between the test instruction’s graph embedding and each candidate tool’s graph embedding. The retrieval process is formulated as:

$$\mathcal{T}'_{\text{ret}} = \{t_j | t_j \in \mathcal{T}_{\text{logic}}, j \in \text{Top-}K(\cos(\mathbf{e}_{q^{\text{test}}}, \mathbf{e}_{t_j}))\} \quad (14)$$

In summary, this two-stage retrieval mechanism synergistically integrates coarse-grained logical pruning and fine-grained graph-enhanced similarity matching, enabling both efficient and accurate tool retrieval even for unseen tools.

5 Experiment

In this section, we conduct comprehensive experiments to verify the effectiveness and efficiency of LoSemB. Specifically, we aim to answer the following questions. **Q1: (Retrieval Quality):** How does LoSemB perform in terms of tool retrieval accuracy compared with state-of-the-art tool retrieval methods across both transductive and inductive settings? **Q2: (Generation Performance):** How does LoSemB perform in downstream generation tasks when compared against state-of-the-art tool retrieval methods? **Q3 (Ablation Studies):** What contribution does each key component of LoSemB

make to the overall performance? **Q4 (Noise Analysis):** How robust is LoSemB to incorrect instruction-tool pairs in the training data compared to baseline methods? (Note that for an in-depth evaluation of LoSemB, additional experiments on efficiency analysis, backbone analysis, parameter sensitivity and case study are presented in Appendix C.)

5.1 Experimental Setup

Datasets. We evaluate the performance of LoSemB on three tool learning benchmarks: ToolBench [36], UltraTool [21], and API-Gen [24]. Specifically, ToolBench is a large-scale synthetic benchmark containing over 16,000 tool collections with approximately 47,000 unique API interfaces across three distinct scenarios. API-Gen provides 3,673 executable APIs spanning 21 categories. In contrast to these synthetic benchmarks, UltraTool comprises 5,824 real-world instruction examples across 22 diverse domains, incorporating 2,032 tools. More details can be found in Appendix B.1.

Baselines. We categorize all the baselines into three groups: (i) Zero-shot LLM Inference. We evaluate several foundational models, including GPT-3.5-turbo and GPT-4o-mini [35]. Given the different input constraints of LLMs, we select 400 and 2,000 tools, respectively, to incorporate into the prompt, where the LLM directly selects the relevant tools. (ii) Training-free Tool Retrieval Methods. We compare against several retrieval baselines that require no domain-specific training: BM25 [40], a sparse retrieval method based on lexical matching; Ada Embedding, a dense retrieval approach that uses OpenAI’s text-embedding-ada-002 model to encode instructions and tool documents for similarity matching; all-MiniLM-L6-v2 [48], a pre-trained sentence-transformers model that computes semantic similarity between instructions and tools; and Re-Invoke [9], an unsupervised baseline that enhances retrieval through query rewriting and document expansion to bridge the semantic gap between instructions and tool descriptions. (iii)

Training-based Tool Retrieval Methods. We compare against several methods that require domain-specific training on instruction-tool pairs: EmbSim, a retriever directly fine-tuned on instruction-tool pairs to learn task-specific embeddings; InstRet (Instruction-Augmented Retrieval), inspired by Re-Invoke, which enriches the training data by integrating tool documents with instructions to create augmented instruction-tool pairs for fine-tuning; and RerankRet (Reranking-Augmented Retrieval), a two-stage retrieval pipeline that first retrieves candidate tools using fine-tuned embeddings and then applies a cross-encoder reranker for final selection.

Evaluation Metrics. For retrieval quality assessment, we use $Recall@K$ and $Precision@K$ with $K \in \{3, 7\}$. We do not include nDCG as it is not well-suited for tool retrieval scenarios [32, 49]. In tool retrieval tasks, tool relevance is binary, and the order of retrieved tools does not affect performance. For downstream generation performance, we adopt metrics from StableToolBench [14], including *Pass Rate*, which measures whether the model successfully completes a given task, and *Win Rate*, which employs an LLM judge to determine which approach produces superior results. Here, we conduct comparisons between outputs generated using baseline-retrieved tools and those generated using LoSemB-retrieved tools.

Implementation Details. We conduct experiments under two settings. In the transductive setting, we filter the test set to remove instructions involving tools not present in the training data, ensuring all tools are included in the training set. In the inductive setting, we randomly select 10%, 20%, and 30% of test tools as unseen tools and exclude them from the training data. Within LoSemB, retrievers are trained using BERT-base as the backbone model.

5.2 Retrieval Quality (Q1)

To address Q1, we conduct an evaluation of retrieval accuracy by comparing various baseline methods with LoSemB across five benchmark datasets. Table 2 evaluates performance under both transductive and inductive settings, where the transductive setting contains no unseen tools in the test set, while the inductive setting includes 30% unseen tools. Figure 4 further illustrates the performance degradation trends of training-based tool retrieval methods across varying unseen tool ratios. Based on these experimental results, we derive the following key observations.

Obs. 1. Tool retrieval methods enhance zero-shot LLM performance. Direct prompting of LLMs with a partial tool repository, constrained by limited context windows, yields the poorest performance across most datasets, except when the tool repository is small enough to fit entirely within the context. For instance, GPT-4o-mini achieves only 8.67% accuracy on ToolBench(I2). In contrast, incorporating a retriever to search for the most relevant tools from the large repository significantly improves performance. However, its effectiveness may vary across different retrievers. Specifically, while lexical methods like BM25 show unstable results, pre-trained dense embedding models such as Ada Embedding deliver consistent improvements, boosting accuracy to 26.34% on ToolBench (I2).

Obs. 2. Domain-specific training aligns user intent with tool descriptions better than general-purpose retrievers, but exhibits limited generalization to unseen tools. Training-based methods consistently outperform training-free approaches, demonstrating the benefits of domain-specific training. However, when

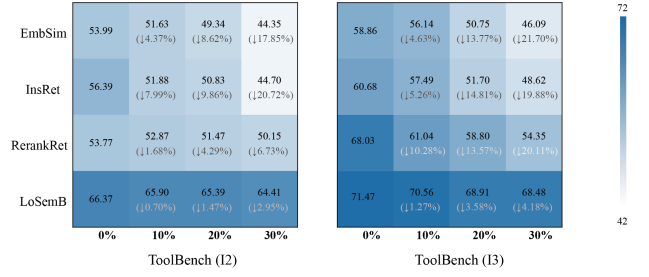


Figure 4: Retrieval results (%) of different baselines across varying percentages of unseen tools. Each cell shows the absolute performance and the relative drop (“↓”) compared to the transductive setting.

moving to the inductive setting with unseen tools, all trained methods exhibit performance degradation across all datasets. For instance, RerankRet achieves solid results on most datasets in the transductive setting, reaching 61.74% on UltraTool, but suffers a significant drop to 48.83% in the inductive setting.

Obs. 3. LoSemB demonstrates superior performance compared to all baselines in both transductive and inductive settings. While training-free methods maintain stable performance across both transductive and inductive settings, they achieve sub-optimal results. Typical training-based retrieval methods also struggle with frequently updated tool repositories, as maintaining high performance typically requires costly retraining. In contrast, by leveraging logical information, LoSemB outperforms all baseline methods by a significant margin across all datasets. Furthermore, as shown in Figure 4, LoSemB exhibits more stable performance as the proportion of unseen tools increases. Specifically, on ToolBench (I3), LoSemB maintains stable performance with relative decreases of merely 1.27%, 3.58%, and 4.18%, respectively.

5.3 Generation Performance (Q2)

Table 3: Generation results (%) of different baselines. The table compares the pass rate and win rate for transductive and inductive settings across ToolBench (I2) and ToolBench(I3).

Dataset	Method	Transductive		Inductive	
		Pass Rate	Win Rate	Pass Rate	Win Rate
I2	Re-invoke	37.25	41.37	37.25	41.52
	EmbSim	44.73	43.84	42.68	42.83
	InstRet	<u>45.71</u>	44.89	43.34	44.13
	RerankRet	43.06	<u>45.13</u>	41.27	<u>44.78</u>
	LoSemB	50.04	NA	48.92	NA
I3	Re-invoke	40.87	40.19	40.87	41.86
	EmbSim	48.16	45.79	45.23	44.65
	InstRet	49.15	47.44	45.08	42.79
	RerankRet	53.72	<u>47.66</u>	<u>47.38</u>	45.58
	LoSemB	57.24	NA	54.27	NA

To investigate the impact of retrieval on generation, we use GPT-4o-mini as the task execution model to evaluate the generation quality of different tool retrieval methods under both transductive and inductive settings. As shown in Table 3, we observe that:

Obs. 4. LoSemB achieves superior generation quality with the highest task completion rate. LoSemB consistently achieves

the highest pass rate across both datasets and settings, reaching 50.04% and 48.92% on ToolBench(I2) in transductive and inductive settings, respectively. Furthermore, all baseline methods exhibit win rates below 50% when compared against LoSemB, indicating that LoSemB produces higher-quality outputs than other baselines. This demonstrates that accurate tool retrieval translates directly into improved downstream task performance.

5.4 Ablation Studies (Q3)

We conduct systematic ablation studies on the core components of LoSemB on ToolBench (I2). We examine two key modules: I. Logic-based Embedding Alignment Module, with two variants including *w/o Instruction Transferring (w/o Ins-Trans)*, which uses only semantic embeddings for test instructions to calculate similarity, and *w/o Tool Transferring (w/o Tool-Trans)*, which directly adopts semantic embeddings for unseen tool nodes; and II., Relational Augmented Retrieval Mechanism, *i.e.*, *w/o Relational Retrieval (w/o Rel-Retrieval)*, which computes similarity scores with all available tools. Each variant was evaluated across different ratios of unseen tools. From the experimental results in Table 4, we can observe that:

Table 4: Ablation results (%) on key modules of LoSemB on ToolBench (I2). "Drop ↓" rows show relative performance degradation (%) compared to the full model.

Variant	0%		10%		20%		30%	
	R@3	P@3	R@3	P@3	R@3	P@3	R@3	P@3
w/o Ins-Trans	78.00	62.32	78.40	62.56	76.69	61.27	75.55	60.04
Drop ↓	3.12	3.20	1.54	1.48	2.34	2.16	2.14	2.18
w/o Tool-Trans	NA	NA	78.57	62.73	76.96	61.56	75.58	59.98
Drop ↓	NA	NA	1.33	1.21	2.00	1.69	2.15	2.28
w/o Rel-Retrieval	72.27	57.22	65.94	52.70	64.90	51.53	55.92	44.25
Drop ↓	10.23	11.12	17.19	17.01	17.36	17.71	27.60	27.91
Full model	80.51	64.38	79.63	63.50	78.53	62.62	77.24	61.38

Table 5: KL divergence between unseen tool embeddings and training data before and after logic-based alignment

Variant	ToolBench (I2)	ToolBench (I3)
w/ Logic-based Alignment	0.001276	0.001132
w/o Logic-based Alignment	0.18183	0.14869

Obs. 5. Each module of LoSemB is critical for both transductive and inductive settings. Our LoSemB achieves the best performance across different proportions of unseen tools. Notably, as the number of unseen tools increases, the contribution of our modules becomes larger, highlighting the crucial role of our approach in addressing inductive scenarios.

To further validate the effectiveness of Logic-based Embedding Alignment in addressing the distributional shift problem identified in Section 3.2. We measure the KL divergence between the embeddings of unseen tools *v.s.* seen tools, comparing the distribution before and after applying the alignment mechanism.

Obs. 6. Logic-based Embedding Alignment effectively resolves the distribution shift. As shown in Table 5, unseen tools

initially exhibit a large distribution gap from seen data, with KL divergence values of 0.18183 (I2) and 0.14869 (I3). Logic-based Embedding Alignment reduces these to 0.001276 and 0.001132, respectively, aligning unseen tool embeddings with the distribution of seen data and enabling accurate retrieval by leveraging logical knowledge from seen data.

5.5 Noise Analysis (Q4)

To analyze the robustness of LoSemB to noisy training data, we evaluate the retrieval performance under scenarios where 10% of instruction-tool pairs in the training set are incorrect, simulating real-world scenarios where tool annotations may be imperfect. We compare the performance of LoSemB against baseline methods on ToolBench (I2) under both clean and noisy training conditions.

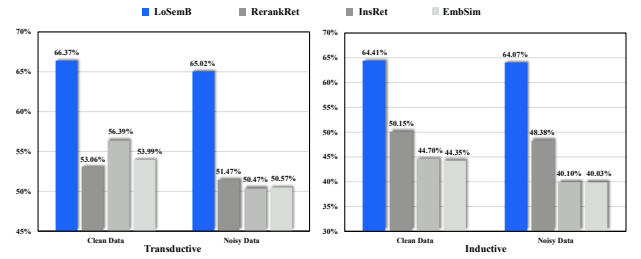


Figure 5: Retrieval results (%) of different baselines under transductive and inductive settings. Clean data indicates no label noise.

Obs. 7. LoSemB demonstrates superior robustness to label noise compared to baseline methods. As shown in Figure 5, LoSemB exhibits minimal performance drops of only 1.35% and 0.34% under 10% label noise, while InsRet suffers absolute drops of 5.92% and 4.60%. This robustness is attributed to the dual verification mechanism in Relational Augmented Retrieval.

6 Conclusion

The main objective of this paper is to tackle the tool retrieval task in the inductive setting. Although retrieval-based methods have shown excellent performance, they still face two challenges: the large distribution shift and the sensitivity of semantic-only retrieval when handling unseen tools. To this end, we propose a logic-based tool retrieval framework, named LoSemB, which integrates latent logical information into the retrieval process. We first employ a logic-based embedding alignment module to mitigate the large distribution shift of unseen tools, and then implement a relational augmented retrieval mechanism that integrates logical pruning and graph-enhanced similarity matching. Extensive experiments in both transductive and inductive settings demonstrate the strong performance of LoSemB. Looking ahead, LoSemB introduces a new paradigm for tool retrieval, paving the way for advanced techniques like sophisticated GNNs and complex graph structures to further enhance tool learning in evolving web tool ecosystems.

7 Acknowledgments

The work described in this paper was fully supported by a grant from the Innovation and Technology Commission of the Hong Kong Special Administrative Region, China (Project No. ITS/263/24FP).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] AI Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. *Claude-3 Model Card* (2024).
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [4] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations*.
- [5] Junzhi Chen, Juhao Liang, and Benyou Wang. 2024. Smurfs: Leveraging Multiple Proficiency Agents with Context-Efficiency for Tool Planning. *arXiv preprint arXiv:2405.05955* (2024).
- [6] Shengyuan Chen, Qinggang Zhang, Junnan Dong, Wen Hua, Jiannong Cao, and Xiao Huang. 2024. Neuro-symbolic entity alignment via variational inference. *arXiv preprint arXiv:2410.04153* (2024).
- [7] Shengyuan Chen, Qinggang Zhang, Junnan Dong, Wen Hua, Qing Li, and Xiao Huang. 2024. Entity Alignment with Noisy Annotations from Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [8] Shengyuan Chen, Chuang Zhou, Zheng Yuan, Qinggang Zhang, Zeyang Cui, Hao Chen, Yilin Xiao, Jiannong Cao, and Xiao Huang. 2025. You Don't Need Pre-built Graphs for RAG: Retrieval Augmented Generation with Adaptive Reasoning Structures. *arXiv:2508.06105 [cs.CL]*
- [9] Yanfei Chen, Jinsung Yoon, Devendra Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. 2024. Re-Invoke: Tool Invocation Rewriting for Zero-Shot Tool Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 4705–4726.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [11] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. In *International Conference on Machine Learning*. PMLR, 11812–11829.
- [12] Vijay Prakash Dwivedi and Xavier Bresson. 2020. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699* (2020).
- [13] Robert L Goldstone and Samuel B Day. 2012. Introduction to “new conceptualizations of transfer of learning”. *Educational Psychologist* 47, 3 (2012), 149–152.
- [14] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *ACL (Findings)*.
- [15] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [16] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in neural information processing systems* 36 (2023), 45870–45894.
- [17] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [18] Zijin Hong, Zheng Yuan, Hao Chen, Qinggang Zhang, Feiran Huang, and Xiao Huang. 2024. Knowledge-to-SQL: Enhancing SQL Generation with Data Expert LLM. *arXiv preprint arXiv:2402.11517* (2024).
- [19] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2024. Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2024).
- [20] Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. 2023. Tool documentation enables zero-shot tool-usage with large language models. *arXiv preprint arXiv:2308.00675* (2023).
- [21] Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, et al. 2024. Planning, Creation, Usage: Benchmarking LLMs for Comprehensive Tool Utilization in Real-World Complex Scenarios. In *ACL (Findings)*.
- [22] Tiffany H Kung, Morgan Cheatham, Arielle Medenilla, Czarina Sillos, Lorie De Leon, Camille Elepaño, Maria Madiaga, Rimel Aggabao, Giesel Diaz-Candido, James Maningo, et al. 2023. Performance of ChatGPT on USMLE: potential for AI-assisted medical education using large language models. *PLoS digital health* 2, 2 (2023), e0000198.
- [23] Yi-Yu Lai, Jennifer Neville, and Dan Goldwasser. 2019. Transconv: Relationship embedding in social networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4130–4138.
- [24] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, et al. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587* (2024).
- [25] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [26] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024).
- [27] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [28] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems* 36 (2023), 43447–43478.
- [29] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2024. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems* 36 (2024).
- [30] Renqiang Luo, Huafei Huang, Ivan Lee, Chengpei Xu, Jianzhong Qi, and Feng Xia. 2025. FairGP: A Scalable and Fair Graph Transformer Using Graph Partitioning. In *Proceedings of the 39th Annual AAAI Conference on Artificial Intelligence*. 12319–12327.
- [31] Renqiang Luo, Huafei Huang, Shuo Yu, Xiuzhen Zhang, and Feng Xia. 2024. FairGT: A Fairness-aware Graph Transformer. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence*. 449–457.
- [32] Suhong Moon, Siddharth Jha, Lutfi Eren Erdogan, Sehoon Kim, Woosang Lim, Kurt Keutzer, and Amir Gholami. 2024. Efficient and scalable estimation of tool representations in vector space. *arXiv preprint arXiv:2409.02141* (2024).
- [33] Feiteng Mu, Yong Jiang, Liwen Zhang, Liuchu Liuchu, Wenjie Li, Pengjun Xie, and Fei Huang. 2024. Query Routing for Homogeneous Tools: An Instantiation in the RAG Scenario. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 10225–10230.
- [34] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, et al. 2022. Large Dual Encoders Are Generalizable Retrievers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 9844–9855.
- [35] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774 [cs.CL]*
- [36] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. [n.d.]. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations*.
- [37] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2024. Towards completeness-oriented tool retrieval for large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1930–1940.
- [38] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2025. Tool learning with large language models: A survey. *Frontiers of Computer Science* 19, 8 (2025), 198343.
- [39] Ievgen Redko, Emilie Morvant, Amaury Habrard, Marc Sebban, and Younès Bennani. 2020. A survey on domain adaptation theory: learning bounds and theoretical guarantees. *arXiv preprint arXiv:2004.11829* (2020).
- [40] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [41] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [42] Chen Shengyuan, Yunfeng Cai, Huang Fang, Xiao Huang, and Mingming Sun. 2023. Differentiable neuro-symbolic reasoning on large-scale knowledge graphs. *Advances in Neural Information Processing Systems* 36 (2023).
- [43] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjing Wang, and Yu Sun. 2021. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 1548–1554.
- [44] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shueb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *TRANSACTIONS ON MACHINE LEARNING RESEARCH* (2022).

- [45] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [46] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [47] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2024. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439* (2024).
- [48] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems* 33 (2020), 5776–5788.
- [49] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. 2013. A theoretical analysis of NDCG type ranking measures. In *Conference on learning theory*. PMLR, 25–54.
- [50] Patrick H Winston. 1980. Learning and reasoning by analogy. *Commun. ACM* 23, 12 (1980), 689–703.
- [51] Feng Xia, Ciyuan Peng, Jing Ren, Fahih Gozi Febrinanto, Renqiang Luo, Vidya Saikrishna, Shuo Yu, and Xiangjie Kong. 2025. Graph Learning. *Foundations and Trends in Signal Processing* 19, 4 (2025), 371–551. doi:10.1561/2000000137
- [52] Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. 2025. When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation. *arXiv preprint arXiv:2506.05690* (2025).
- [53] Yilin Xiao, Chuang Zhou, Qinggang Zhang, Bo Li, Qing Li, and Xiao Huang. 2025. Reliable Reasoning Path: Distilling Effective Guidance for LLM Reasoning with Knowledge Graphs. *arXiv:2506.10508 [cs.CL]* <https://arxiv.org/abs/2506.10508>
- [54] Yilin Xiao, Chuang Zhou, Yujing Zhang, Qinggang Zhang, Su Dong, Shengyuan Chen, Chang Yang, and Xiao Huang. 2026. LAG: Logic-Augmented Generation from a Cartesian Perspective. *arXiv:2508.05509 [cs.CL]* <https://arxiv.org/abs/2508.05509>
- [55] Qiancheng Xu, Yongqi Li, Heming Xia, and Wenjie Li. 2024. Enhancing Tool Retrieval with Iterative Feedback from Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 9609–9619.
- [56] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. 2025. Data-centric artificial intelligence: A survey. *Comput. Surveys* 57, 5 (2025), 1–42.
- [57] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. *arXiv preprint arXiv:2501.13958* (2025).
- [58] Qinggang Zhang, Junnan Dong, Hao Chen, Daochen Zha, Zailiang Yu, and Xiao Huang. 2024. Knowgpt: Knowledge graph based prompting for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 6052–6080.
- [59] Qinggang Zhang, Zhishang Xiang, Yilin Xiao, Le Wang, Junhui Li, Xinrun Wang, and Jinsong Su. 2025. FaithfulRAG: Fact-Level Conflict Modeling for Context-Faithful Retrieval-Augmented Generation. *arXiv preprint arXiv:2506.08938* (2025).
- [60] Yuxiang Zhang, Xin Fan, Junjie Wang, Chongxian Chen, Fan Mo, Tetsuya Sakai, and Hayato Yamana. 2024. Data-Efficient Massive Tool Retrieval: A Reinforcement Learning Approach for Query-Tool Alignment with Language Models. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 226–235.
- [61] Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. 2024. Dense text retrieval based on pretrained language models: A survey. *ACM Transactions on Information Systems* 42, 4 (2024), 1–60.
- [62] Huachi Zhou, Jiahe Du, Chuang Zhou, Chang Yang, Yilin Xiao, Yuxuan Xie, and Xiao Huang. 2025. Each Graph is a New Language: Graph Learning with LLMs. *arXiv preprint arXiv:2501.11478* (2025).
- [63] Luyao Zhuang, Shengyuan Chen, Yilin Xiao, Huachi Zhou, Yujing Zhang, Hao Chen, Qinggang Zhang, and Xiao Huang. 2025. LinearRAG: Linear Graph Retrieval Augmented Generation on Large-scale Corpora. *arXiv preprint arXiv:2510.10114* (2025).

A Related Work

A.1 Tool Retrieval

Recently, LLMs have shown excellent abilities in many tasks. Meanwhile, it becomes increasingly vital to equip LLMs with external tools [11, 33, 54]. In this regard, tool retrieval plays a crucial role, with two main research directions emerging:

Token-based methods. Token-based methods [16, 47] represent each tool as a specific token and generate the relevant tools.

For example, ToolGen [47] implements a comprehensive four-stage process: tool virtualization, tool memorization, retrieval training, and end-to-end agent-tuning. This approach involves a resource-intensive training process. When tools are integrated as tokens, additional tokens must be added and their corresponding documentation fine-tuned into the model’s parameters.

Retrieval-based methods. Retrieval-based methods [53, 57, 59, 63] search relevant tools from the large tool repositories. These methods include both sparse (e.g., BM25 [40]) and dense retrievers, mainly using PLMs. A number of studies have built upon this baseline method [18, 52]. For example, ToolRetriever [36] uses instruction-tool pairs to train PLMs on domain-specific data. QTA [60] addresses low-resource scenarios by using direct preference optimization (DPO) to train LLMs. However, these transductive approaches face challenges from distribution shifts and the vulnerability of similarity-based retrieval in inductive settings. Although Re-Invoke [9] proposes a zero-shot tool retrieval framework, its performance is worse than supervised approaches.

A.2 Graph Convolutional Networks

In recent years, several convolutional neural network architectures for learning over graphs have been proposed [58, 62], which have been shown to effectively model relationship features, relational reasoning [6, 8, 19, 42], and combinatorial generalization [7, 30, 31, 51]. For instance, TransConv [23] further advances this paradigm by incorporating textual interactions between user pairs in social networks. UniMP [43] conceptually unifies feature propagation and label propagation through a unified message passing framework. These GNN methods demonstrate effectiveness in semi-supervised classification tasks. Therefore, we adopt GNNs to extract logical features. In this work, we build upon LightGCN [17], which is particularly suitable for capturing logical information between instructions and tools. Specifically, our approach combines logical features and text features to achieve embedding alignment for unseen tools through feature propagation and aggregation.

B Implementation Details

B.1 Data Statistics

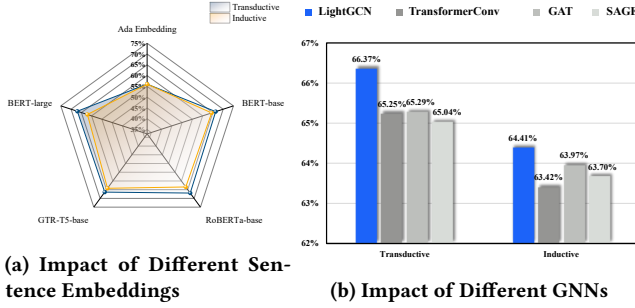
To accurately assess the impact of unseen tools on tool retrieval performance, we conducted experiments under both transductive and inductive settings. In the transductive setting, we filtered the test set to remove instructions involving tools not present in the training data. For the inductive setting, we randomly selected 10%, 20%, and 30% of test tools as unseen tools and excluded them from the training data. The detailed statistics for both settings across all datasets are presented in Table 6.

B.2 Hyperparameters

We set the training epochs for the retriever to 5 with a learning rate of 2×10^{-5} and warmup steps of 500. We carefully tune the layer number L among {2, 3, 4}, parameter J among {3, 4, 5, 6, 7}, and parameter I among {2, 3, 4, 5}. For the GCN component, we set the training epochs to 200 with a learning rate of 1×10^{-3} and a weight decay of 1×10^{-4} .

Table 6: Dataset statistics on ToolBench, UltraTool, and APIGen datasets across varying proportions of unseen tools. Note that the test set remains consistent across all ratios.

Type	Ratio	ToolBench (I1)		ToolBench (I2)		ToolBench (I3)		UltraTool		APIGen	
		Instruction	Tool	Instruction	Tool	Instruction	Tool	Instruction	Tool	Instruction	Tool
Train	0%	86,643	10,439	84,207	13,142	25,044	1,605	5,682	2,032	59,800	3,579
	10%	84,355	10,330	82,104	13,070	19,095	1,573	4,163	2,020	55,272	3,541
	20%	82,136	10,221	79,100	12,998	14,783	1,541	3,153	2,009	51,241	3,502
	30%	80,616	10,112	76,795	12,926	11,261	1,509	2,490	1,998	47,403	3,464
Test	all	792	1,089	568	720	216	317	149	112	200	386

**(a) Impact of Different Sentence Embeddings****(b) Impact of Different GNNs****Figure 6: (a) Impact of different sentence embeddings, (b) Impact of GNNs. All results are evaluated on ToolBench (I2).**

C Additional Experiments

C.1 Efficiency analysis (Q5)

To address Q5, we compare the efficiency of LoSemB against training-based baselines on ToolBench (I2) under the transductive setting where 30% of tools are unseen. Our efficiency analysis spans three stages: (1) the indexing stage; (2) the incremental indexing (Incr. Indexing) stage for processing and incorporating unseen tools; and (3) the inference stage for retrieval. We observe the following:

Table 7: Efficiency and performance comparison of training-based baselines on ToolBench (I2) with 30% unseen tools. Notably, Perf. denotes the average retrieval performance across R@3, R@7, P@3, and P@7.

Method	Indexing (h)	Incr. Indexing (min)	Inference (ms)	Perf. (%)
EmbSim	8.42	668.33	11.37	53.99
InsRet	14.53	930.33	21.14	56.39
RerankRet	24.86	2148.80	4373.82	53.77
LoSemB	8.58	5.66	12.38	64.41

Obs. 8. LoSemB achieves superior efficiency and effectiveness in handling unseen tools. As shown in Table 7, LoSemB exhibits comparable initial indexing time (8.58h vs. 8.42h) and inference time (12.38ms vs. 11.37ms) to the traditional EmbSim approach. However, the key distinction emerges when processing unseen tools through incremental indexing. EmbSim requires computationally expensive retriever retraining, consuming 668.33 minutes (over 11 hours). In contrast, LoSemB leverages its logic-guided semantic bridging approach to process unseen tools in merely 5.66 minutes—a 118× speedup. This dramatic efficiency gain, coupled

with a substantial performance improvement (64.41% vs. 53.99%), makes LoSemB the most practical choice for deployments demanding strong performance in frequently updated tool repositories, where scalability and cost control are critical.

C.2 Impact of Sentence Embeddings (Q6)

In this section, we study the effectiveness of different sentence embeddings on retrieval performance in LoSemB. We compare Ada Embedding, BERT-base [10] (110M parameters), RoBERTa-base [27] (125M parameters), GTR-T5-base [34] (110M parameters), and BERT-large-uncased [10] (336M parameters) on the ToolBench (I2) dataset. Notably, except for Ada Embedding, all other models are retriever backbones fine-tuned on instruction-tool pairs. The retrieval performance is evaluated under both transductive and inductive (30% unseen tools) settings.

Obs. 9. BERT-base demonstrates the best trade-off between performance and robustness to unseen tools among all evaluated sentence embeddings. Based on the findings presented in Table 6 (a), we observe that while all sentence embedding models achieve relatively comparable performance, BERT-base stands out by demonstrating both strong effectiveness in the transductive setting and superior robustness in the inductive setting with unseen tools. Notably, even Ada Embedding without domain-specific training achieves competitive results, demonstrating the effectiveness of our framework. Consequently, we adopt BERT-base as the default embedding model in all experiments.

C.3 Impact of Different GNNs (Q7)

In this section, we investigate the impact of different GNN architectures on the retrieval performance of LoSemB. We compare four representative GNN models: GraphSAGE [15], GAT [46], TransformerConv [12], and LightGCN [17] on the ToolBench (I2) and ToolBench (I3) datasets. The evaluation is conducted under both transductive and inductive (30% unseen tools) settings.

Obs. 10. LightGCN consistently achieves the best performance across different settings among all evaluated GNN architectures. Based on the findings presented in Figure 6 (b), we observe that while all GNN architectures demonstrate competitive performance, LightGCN exhibits superior results in the majority of experimental configurations. However, LightGCN’s simplified design, which removes feature transformation and nonlinear activation while focusing on neighborhood aggregation, proves particularly effective for capturing tool-instruction relationships in our

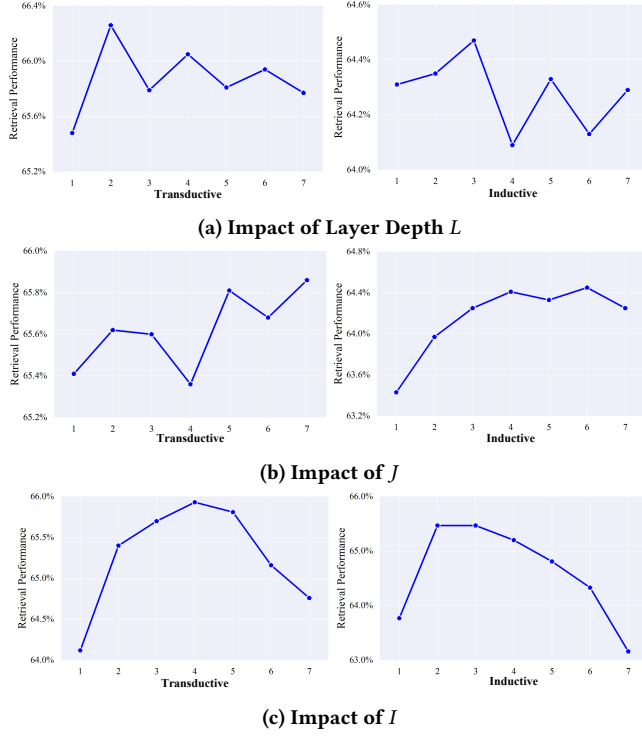


Figure 7: Parameter analysis of LoSemB. (a) shows the dependency of LoSemB performance on layer depth L . (b) illustrates the influence of J . (c) examines the effect of I .

logic-guided semantic space. Consequently, we adopt LightGCN as the default GNN architecture in all experiments.

C.4 Parameter Sensitivity (Q8)

We investigate the impact of key hyperparameters in LoSemB. Experiments are evaluated with the average scores of R@3, R@7, P@3, and P@7 in transductive and inductive settings on ToolBench (I2).

Obs. 11. Moderate layer depth yields optimal performance, while excessive depth leads to over-smoothing and degraded retrieval quality. Based on the findings presented in Figure 7 (a), we observe that incorporating logical information to generate graph embeddings yields superior performance compared to relying solely on semantic embeddings. This demonstrates the effectiveness of our logic-guided semantic bridging approach. However, deeper layers enable richer neighborhood information aggregation, excessively large L values can be harmful to the effectiveness of LoSemB due to the over-smoothing problem. Consequently, we adopt a moderate layer depth in all experiments to balance the benefits of multi-hop reasoning with the preservation of feature distinctiveness.

Obs. 12. Obtaining logical information from a moderate number of similar instructions effectively improves performance, while excessive nodes introduce irrelevant information and degrade results. Based on the findings presented in Figure 7 (b), we observe that obtaining logical information from similar instructions and tools is effective, with performance improving as the number increases. This demonstrates the effectiveness of our logical feature transfer mechanism. However, more nodes

are added, the improvement slows down and eventually drops, as less relevant nodes bring in unhelpful information that interferes with the model’s judgment. Consequently, we adopt a moderate J value to fully leverage logical information from similar instructions while avoiding noise introduction.

Obs. 13. The logical tool set size must balance recall and precision, as oversized sets introduce functionally mismatched tools. Based on the findings presented in Figure 7 (c), we observe that as I increases, the logical tool set $\mathcal{T}_{\text{logic}}$ selected based on the most similar instructions grows larger, initially improving performance. This indicates that expanding the candidate set helps increase the likelihood of recalling relevant tools. However, once I becomes too large, performance begins to decline as larger sets tend to include tools that are textually similar but functionally different, introducing errors that negatively affect performance. Consequently, we adopt an appropriate I value to achieve the optimal balance between expanding the candidate scope and maintaining functional relevance.

C.5 Case Study on Retrieved Tools (Q9)

Table 8: Selected example from ToolBench showing top-1 tool retrieval results.

Instruction: I’m planning a family gathering and I want to make some non-alcoholic cocktails for the kids. Can you suggest some fun and colorful mocktail recipes? Also, provide me with the contact details of a brewery that offers non-alcoholic beer options.
EmbSim: [Food, Tasty, recipes/detail]
LoSemB: [Food, The Cocktail DB, Detailed Cocktail Recipe by ID]
Label: [Food, The Cocktail DB, Detailed Cocktail Recipe by ID]

In this section, we demonstrate how tool retrieval benefits from LoSemB using the ToolBench dataset as examples. Our analysis focuses on two key aspects: the logic-based embedding alignment module and the relational augmented retrieval mechanism.

Obs. 14. The synergy between logic-based embedding alignment and relational augmented retrieval enables LoSemB to overcome the limitations of semantic-only methods. Based on the case presented in Table 8, we observe that EmbSim retrieves a generic recipe tool based on textual similarity to “recipes,” failing to capture the specific requirement for cocktails. In contrast, LoSemB correctly identifies “The Cocktail DB” tool through two complementary mechanisms working in tandem. First, the logic-based embedding alignment module learns the semantic relationships among instructions involving terms like “mocktail,” “cocktail,” and “recipes” through the instruction-tool graph structure, enabling the model to understand that mocktail instructions require cocktail-specific tools rather than general recipe databases. Second, the relational augmented retrieval mechanism identifies similar instructions within the cocktail domain (e.g., “I want to surprise my partner with a special cocktail. Can you suggest a unique and romantic cocktail recipe?”) and leverages their associated tools to narrow down the search space to cocktail-relevant tools. This demonstrates the practical value of our approach in real-world tool recommendation scenarios where domain-specific understanding is crucial.